TD4 noté – Topological Data Analysis

Raphael des Boscs Karel Nyobé

22nd October, 2026

Contents

1 Presentation of the code **1**
1.1 Reading the filtration file 1
1.2 Computing the boundary matrix 1
1.3 Reduction algorithm and computation of the barcode 2
1.4 Experiments 3

2 How to run it **3**

3 Summary of the complexities **4**

4 Detailed answers to the questions of the TD **4**

1 Presentation of the code

1.1 Reading the filtration file

As we decided to implement our algorithm in Python, we rewrote the function that reads the filtration text file. The data is stored in a list of a custom object called `Simplex`, holding 3 fields for the value of the filtration, the dimension of the simplex and the vertices of the simplex. We already sort it by filtration value (and dimension in case of equality) for convenience.

Functions

- `read_data(filename) -> list(Simplex)`

1.2 Computing the boundary matrix

1.2.1 Dense and naïve version

We first implemented a naïve version using a dense representation of the boundary matrix. In this version, `compute_boundary_naive` computes the dense boundary matrix of the list of simplices.

This computation of the boundary matrix is correct, and it works on the simple example filtration given in the TD, but it is far too memory-expensive in practice. Indeed, with N simplices, the boundary matrix has size N^2: in the `filtration_A` example ($\sim$430k simplices), if each int is encoded in 8 bytes, that represents $N^2 \times 8 \approx 1.5\,\text{TB}$ memory, which is completely infeasible.

Moreover, because of the two nested loops ranging over the whole list of simplices, the complexity is $O(N^2)$, which would be prohibitive even ignoring the memory issue.

Functions

- `compute_boundary_naive(v) -> list(list(int))`

1.2.2 Sparse and optimized version

Considering the enormous memory cost of the dense representation, we switched to a sparse representation, where the boundary matrix column only contains the indices of the vertices of the simplices represented by the column.

Moreover, we optimize the algorithm (using a dictionary that maps the facets of a simplex to their indices) by directly decomposing the simplex in its facets and adding them to the boundary matrix. This way, we only range over the list of simplices once, and the complexity of the algorithm is $O(N * k)$, where k is the maximal dimension of the simplices of the simplicial complex. In our examples, $k \leq 3$, so the computation is in $O(N)$.

Functions

- `compute_boundary(v) -> (list(set(int)), list(int), list(float))`

1.3 Reduction algorithm and computation of the barcode

1.3.1 Dense and naïve version

Following the approach used for the boundary matrix, we first tested a naïve version with a dense representation. Thus, we wrote a naïve Gaussian pivot algorithm function reducing the dense boundary matrix (`reduction_algorithm_naive`). Because of the three nested loops (we range twice over the columns, and each addition of column ranges over the whole column), we have a complexity $O(N^3)$.

Once again, the algorithm is correct and effectively reduces the matrix for simple filtrations, but the complexity (both in time and memory) is prohibitive for larger examples.

Functions

- `last_one(x) -> int`
- `reduction_algorithm_naive(M) -> list(list(int))`

1.3.2 Sparse and optimized version

Thus, we rewrote the `reduction_algorithm` using the sparse representation. Let's consider the case of a sparse boundary matrix, meaning that the maximal number n of non-zero values in each column is low compared to the number of simplices ($n \ll N$). Not only does this representation allow us to have an acceptable memory cost of $O(N)$ for such a sparse matrix, but it also takes our compute cost from $O(N^3)$ to $O(N)$ in practice because

(i) the addition of two columns is in $O(n_{c_i} + n_{c_j})$ with n_{c_i}, n_{c_j} being the number of non-zero values in C_i and C_j, so $\sim O(1)$ for a sparse matrix where $n_{c_i}, n_{c_j} \ll N$

(ii) for the same reason of sparsity, have to subtract around $n \ll N$ columns from a given column to reduce it (this is not always true for N very large, see question 7)

(iii) a pivot array stores, for each row, the column whose lowest 1 is in this row, so we find the column to subtract in $O(1)$ instead of ranging over all the previous columns

However, in the worst case of a dense matrix, we would have $n_{c_j} \approx N$ for all C_j, giving us back a complexity of $O(N^3)$, and a memory cost of $O(N^2)$.

Note. Initially, we computed the addition of two columns ourselves using a custom `fun_xor` function. However, as we observed that our code was relatively slow, we switched to an implementation using the Python built-in `set` type, which relies on C. The compute time improved a lot, from about a factor 10. We still include the deprecated `fun_xor` function in our code for the record.

We then compute the barcode, which stores the birth and death of each homological feature of the filtration. We only run through the simplices once, so the complexity is $O(N)$.

Functions

- `fun_xor(l, m) -> list(int)` *(deprecated)*
- `reduction_algorithm(c) -> list(set(int))`
- `barcode(c, d, f, output_name) -> None`

1.4 Experiments

Thanks to `filtration_spheres` and `filtration_balls`, we generate simplicial filtrations for the d-sphere and $(d+1)$-ball up to $d = 10$.

Functions

- `filtrations_spheres() -> None`
- `filtrations_balls() -> None`
- `list_to_string(l) -> str`

2 How to run it

All filtrations are stored in the folder `filtrations`. The `main()` function already contains everything necessary to compute the barcode of a simplicial filtration written in a source text file, and output it in an output text file. All output barcode files are located in the folder `output`.

```
def main():
    # start = time.perf_counter()
    c, d, f = compute_boundary(read_data("filtrations/filtration_A.txt"))
    barcode(reduction_algorithm(c), d, f, "filtration_A")
    # elapsed = time.perf_counter() - start
    # print(f"{elapsed:.4f} s")
```

1. To conduct time measurements, uncomment the lines in the `main()` function
2. To change the filtration on which the barcode is computed, update the source filtration file name and the output file name

3 Summary of the complexities

For more details on the complexity of the `compute_boundary` and `reduction_algorithm` functions (and their naive counterparts), see section 1 – Presentation of the code (1.2 – 1.3).

Let N be the number of simplices in the simplicial complex and k be the maximal dimension of the simplices of the complex. We talk about a *Sparse Case* whenever the number of non-zero values in each column of the matrix is $\ll N$.

Function	**Time complexity**	**Memory complexity**
`read_data`	$O(N \log N)$	$O(N \cdot k)$
`compute_boundary_naive`	$O(N^2)$	$O(N^2)$
`compute_boundary`	$O(N \cdot k)$. For $k \ll N$, $O(N \cdot k) \approx O(N)$.	$O(N \cdot k)$
`reduction_algorithm_naive`	$O(N^3)$	$O(N^2)$
`reduction_algorithm`	*Sparse Case*, $O(N)$. *Worst case*, $O(N^3)$.	*Sparse Case*: $O(N \cdot k)$ *Worst case*: $O(N^2)$
`barcode`	$O(N)$	$O(N)$

Table 1: Time and memory complexity of the main functions.

4 Detailed answers to the questions of the TD

Question 2. See 1.3.1.

Question 3. See 1.3.2.

Question 5. We generated the filtrations for the d-spheres and $(d+1)$-balls with the functions `filtration_spheres` and `filtration_balls`, while we wrote the filtrations for the Moebius band, the Klein bottle, the projective plane and the torus by hand.

Here are the drawings of our simplicial complexes for those last 4 examples.

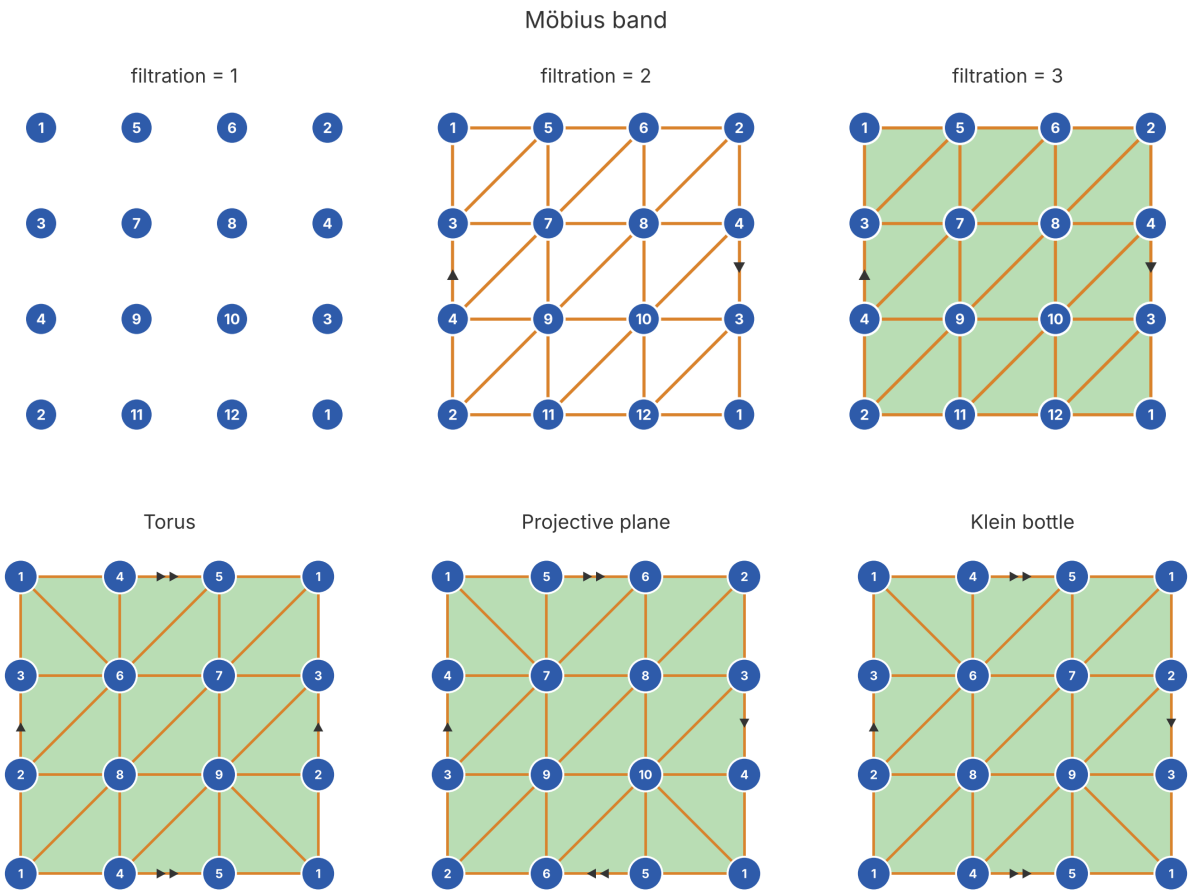


Figure 1: Filtration of the Moebius band (top): vertices have filtration value 1, edges 2 and triangles 3. The torus, the projective plane and the Klein bottle (bottom) follow the same filtration; only their full triangulation is shown.

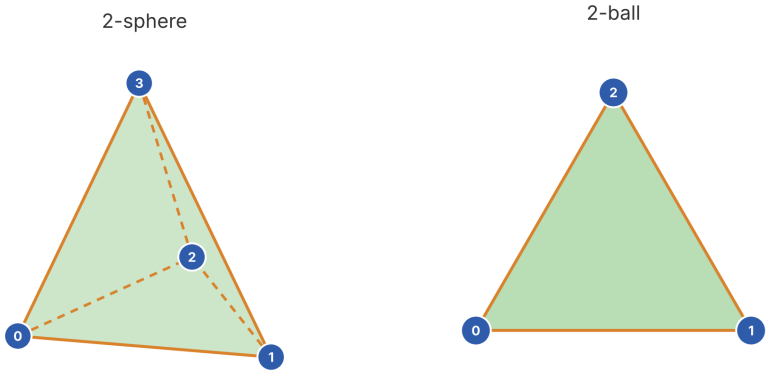


Figure 2: The 2-sphere and the 2-ball, examples of the filtrations generated by `filtration_spheres` and `filtration_balls`.

Question 6. We generate the barcode for the filtrations written at question 5. Reading the barcodes (which can be found in the `output` folder), we observe the following set of simplices with infinite persistence for each filtration:

- **d-sphere.** 1 simplex of dimension 0, 1 simplex of dimension d
- **$(d+1)$-ball.** 1 simplex of dimension 0.
- **Torus.** 1 simplex of dimension 0, 2 simplices of dimension 1, 1 simplex of dimension 2
- **Möbius.** 1 simplex of dimension 0, 1 simplex of dimension 1
- **Projective Plane.** 1 simplex of dimension 0, 1 simplex of dimension 1, 1 simplex of dimension 2.
- **Klein Bottle.** 1 simplex of dimension 0, 2 simplices of dimension 1, 1 simplex of dimension 2.

This all corresponds to what we saw in class, it.e. the Betti numbers of these homologic figures, computed with coefficients in $\mathbb{Z}/2\mathbb{Z}$.

Question 7. Our time measurements on the provided filtrations are presented in the table below. We measure the time execution from the reading of the source filtration file to the creation of the output barcode file. The number $n_{\max}$ corresponds to the maximal number of non zero values in the columns of the boundary matrix of the filtrations.

Name of the filtration	**Number of simplices (N)**	$n_{\max}$	**Time of execution (s)**
`filtration_A`	428 643	4	259.77
`filtration_B`	108 161	4	6.44
`filtration_C`	180 347	4	14.07
`filtration_D`	2 716 431	4	957.82

Table 2: Execution time of the barcode computation on the provided filtrations.

For all of these filtrations, we have $n_{\max} \ll N$, so we are in the case of a sparse boundary matrix, thus the execution time of the reduction algorithm runs in $O(N)$.

In each of these filtrations, the simplices are of the dimension ≤ 3, so the boundary matrix computation runs in $O(N)$.

Therefore, the time measured (filtration reading, boundary matrix computation, boundary matrix reduction, barcode generation) is

$$O(N \log N) + O(N) + O(N) + O(N) = O(N \log N).$$

We plot runtime$/(N \log N)$ for these 4 examples.

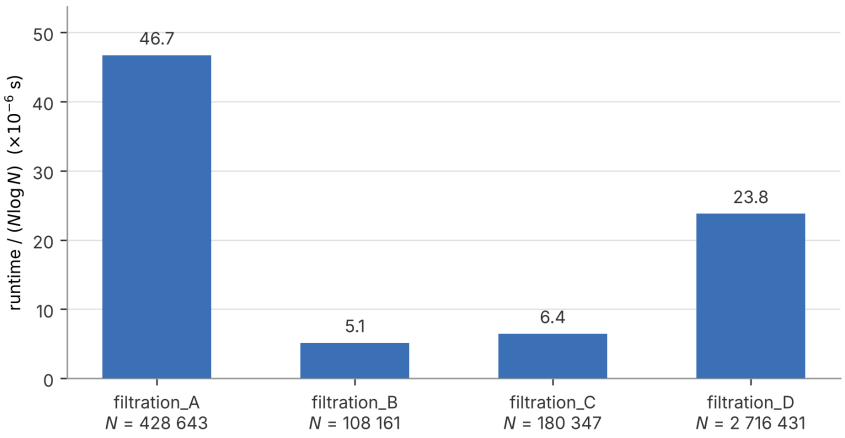


Figure 3: Execution time divided by $N \log N$ for the four provided filtrations.

Although the ratios are globally within one order of magnitude, we don't observe a constant as expected, but we notice that the ratio is higher for `filtration_A` and `filtration_D`, which correspond to the bigger simplicial complexes. One explanation might be that the $n_{\max}$ we computed was on the boundary matrix, but the reduction algorithm progressively populates the columns of the matrix, especially for larger matrices, to the point that the $O(N)$ estimate for the reduction algorithm is not valid anymore.

Question 8.

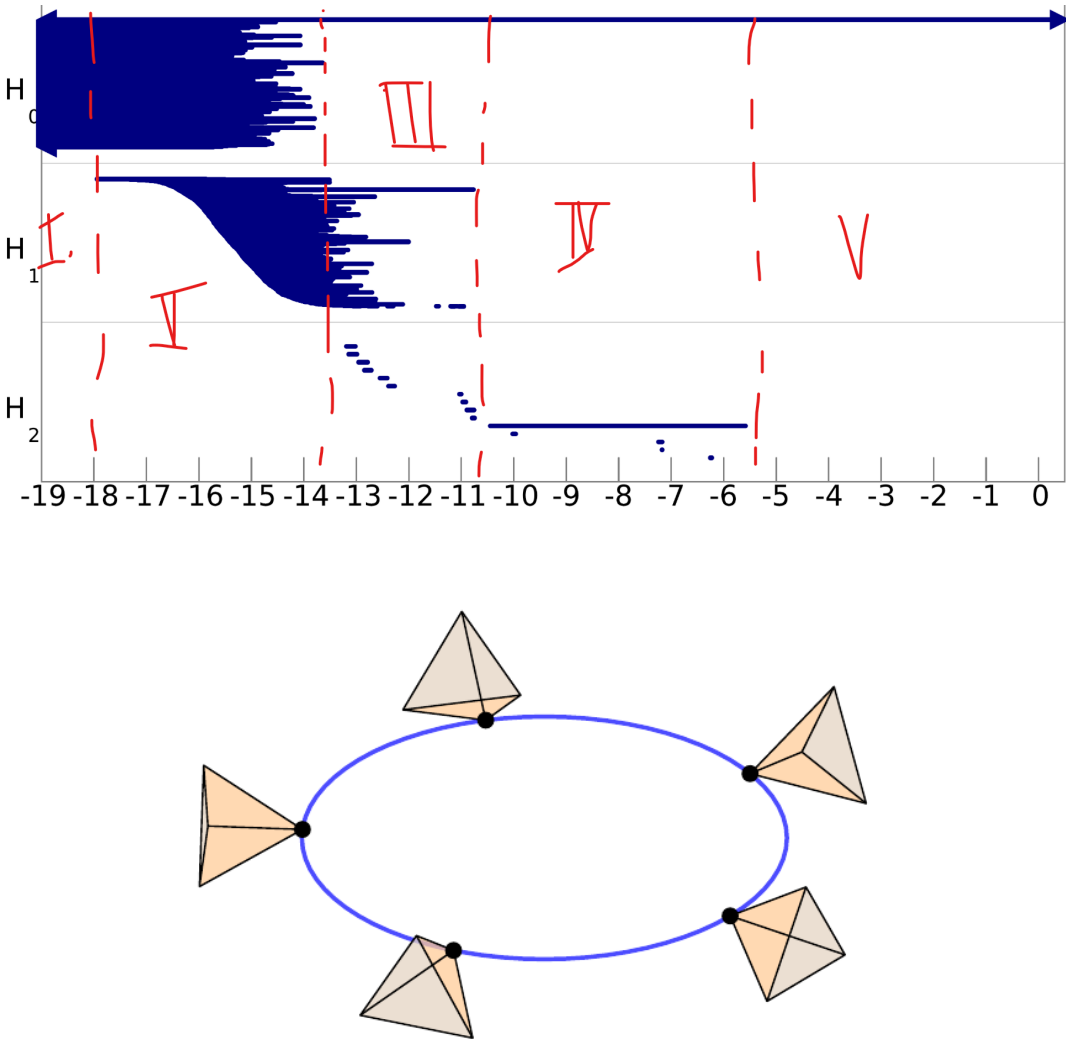


Figure A: Top: annotated barcode of `filtration_A`, with its phases. Bottom: a circle with 5 points (15 in reality) , each point being the apex of a tetrahedron.

The final figure (phase V) has the homology of a point. In phase 4 we have one connected component and spheres that appear (when the last face is added) and disappear (when they are filled with a simplex of dimension 3, which makes the spheres homotopy equivalent to points). The same thing happens in phase 3, except that some future spheres still have several holes, hence the presence of 1-cycles.

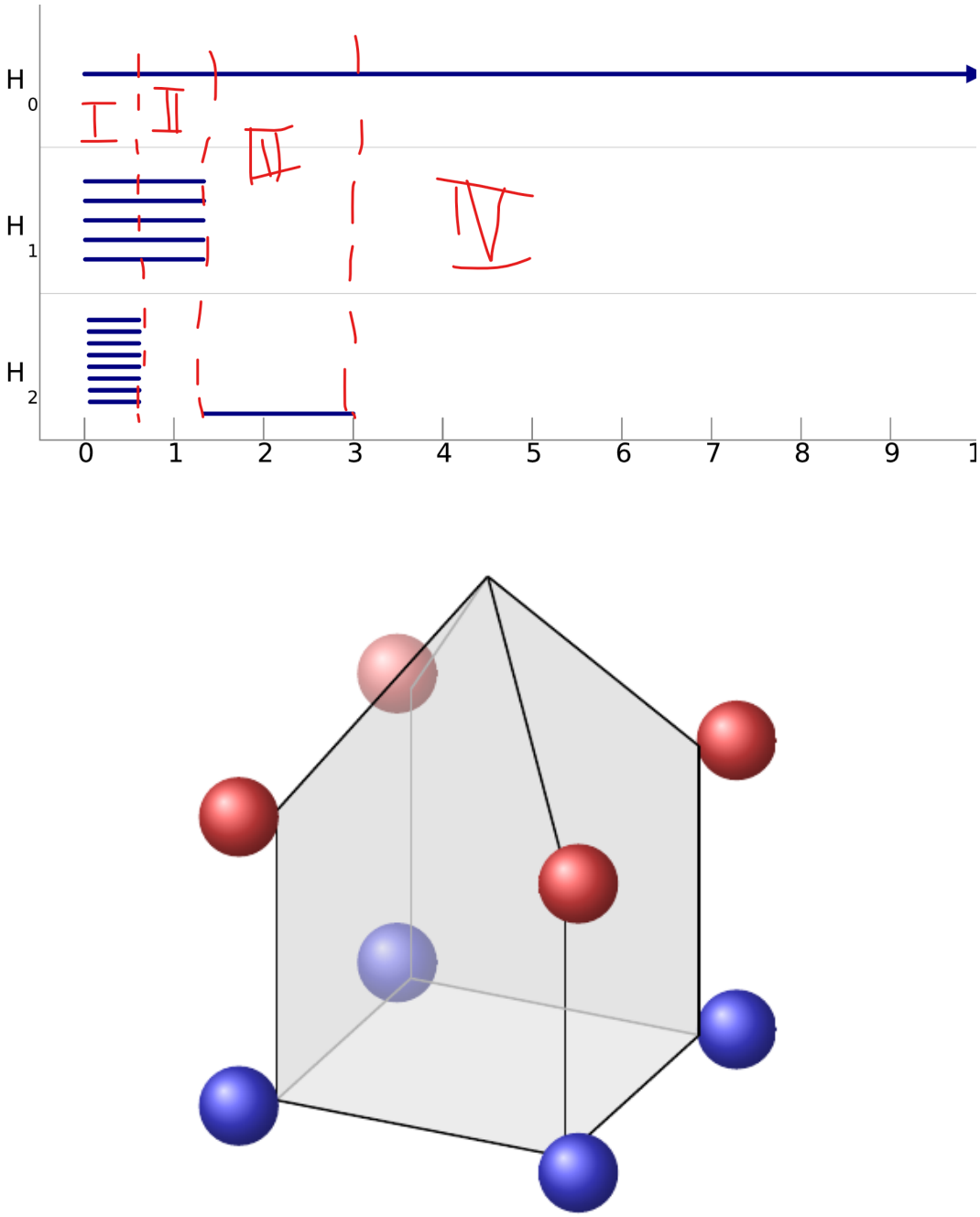


Figure B: Top: annotated barcode of `filtration_B`, with its phases. Bottom: a net (5 holes) enveloping a sphere, with 8 spheres attached to it.

From phase III to IV we go from a sphere to a point (by filling the sphere with simplices of dimension 3). Phases I and II therefore lead to a sphere. We go from phase II (one connected component and five 1-cycles) to phase III (a sphere). We can therefore imagine that phase III corresponds to a net enveloping a sphere (a net with 5 holes). The net is already present in phase I, and 8 spheres are attached to it (one connected component and $|H_2| = 8$). These spheres are filled in to move to phase

II, which makes them homotopy equivalent to points.

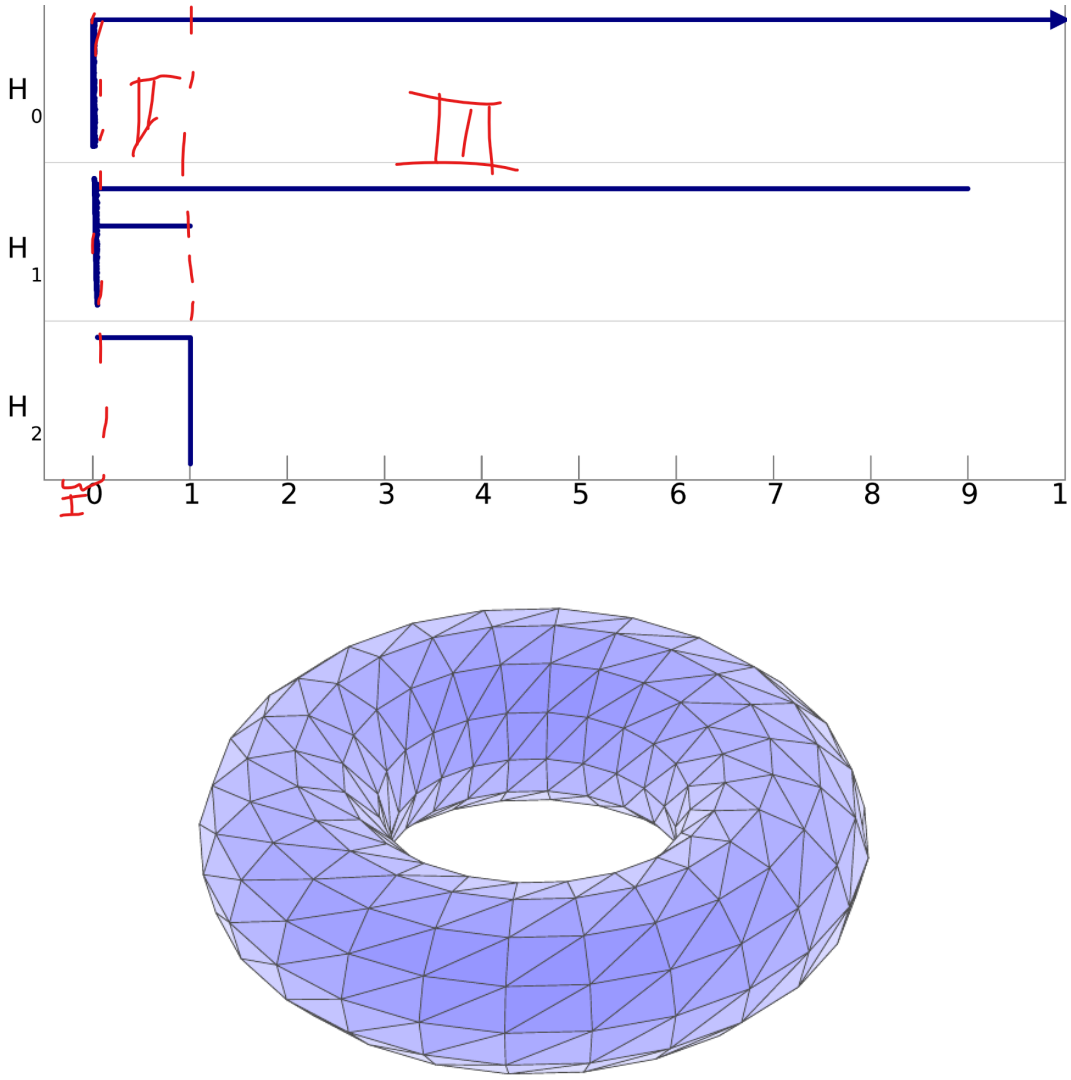


Figure C: Top: annotated barcode of `filtration_C`, with its phases. Bottom: phase III: a circle.

Phase III corresponds to a circle. We go (II $\rightarrow$ III) from a torus to a circle by filling the torus. The torus (phase I) is introduced all at once, with points on the torus, then edges between these points that create 1-cycles, then faces that fill the 1-cycles all around the torus (all at $t = 0$).

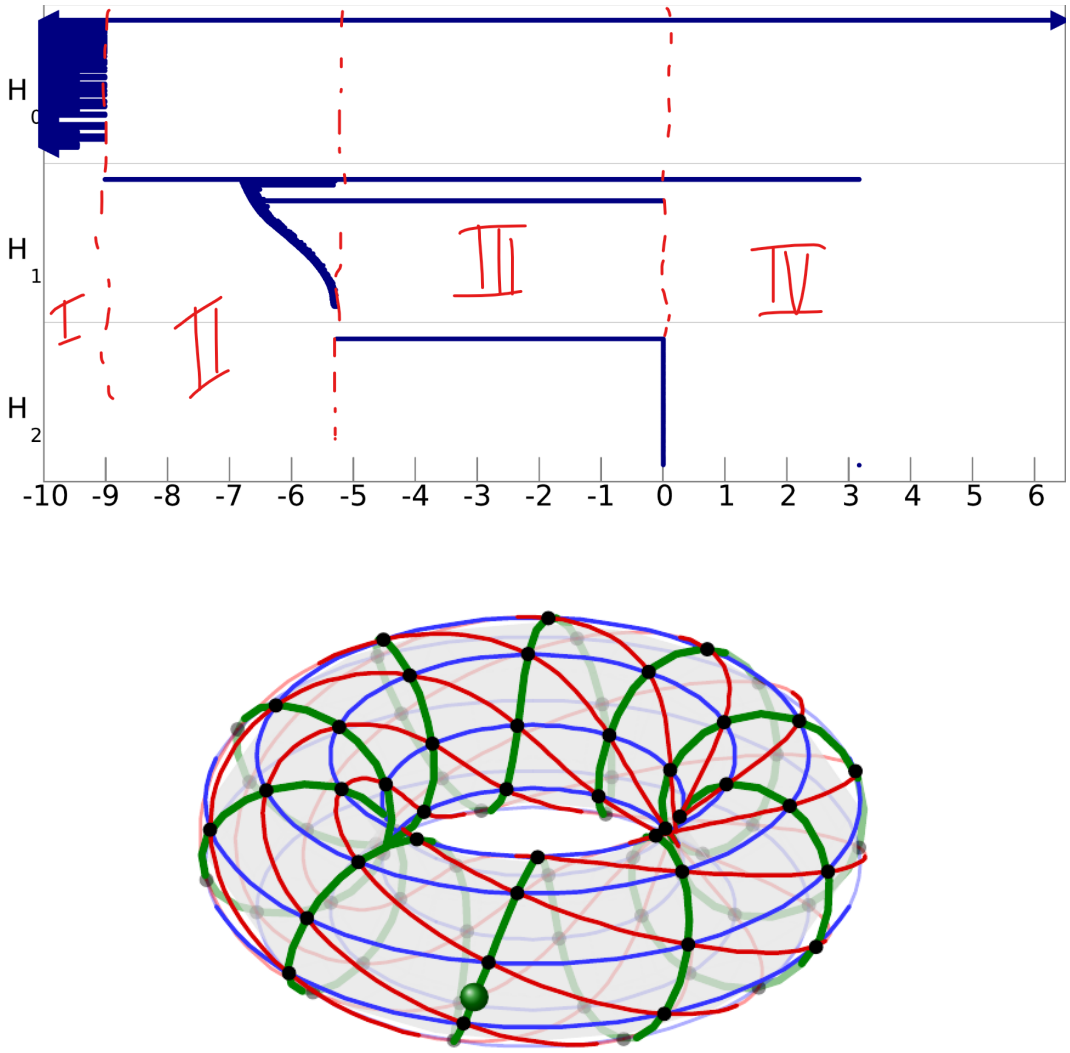


Figure D: Top: annotated barcode of `filtration_D`, with its phases. Bottom: a snake winding around a torus.

From phase III to IV we go from the homotopy class of the torus to that of the circle (typically by filling the inside of the torus to obtain something that retracts onto a circle).

Phases I to II therefore lead to a torus. In phase I there are only simplices of dimension 0 or 1 (to have a 2-simplex we need a 1-cycle, and there is none in phase I). All the points appear at the value -10 (log-scale), and they are then joined by edges that appear one after another. From what precedes, we know that they lie on a torus. Phase 2 begins when they are all connected to one another (a single connected component forming a cycle, so H_1 appears). We can therefore imagine a snake winding around a torus, with points all along it (the green line, with the vertices along it).

During phase 2 (on the right), 1-cycles start to appear (edges between non-consecutive points are added – blue and red edges) and then disappear (the 1-cycles are filled by 2-simplices). Phase 3 begins when we finish filling the surface of the torus. Then nothing happens until phase 4, where the whole interior is filled at once.